

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Meeting AI Agent

Hệ thống Xử lý Audio Cuộc họp

Môn học: Ứng dụng xử lý ngôn ngữ tự nhiên trong doanh nghiệp
Giảng viên: PhD. Nguyễn Hồng Bửu Long
Giảng viên: PhD. Lương An Vinh

Sinh viên thực hiện

22127398 – Nguyễn Văn Minh Thiện

Contents

Tóm tắt	1
1 Giới thiệu	2
1.1 Bối cảnh và động lực	2
1.2 Bài toán NLP	2
1.2.1 Nhận dạng tiếng nói tự động	2
1.2.2 Phân tách người nói	2
1.2.3 Trích xuất thông tin có cấu trúc	3
1.3 Mục tiêu hệ thống	3
1.4 Phạm vi dự án	3
1.5 Đóng góp chính	4
1.6 Cấu trúc báo cáo	4
2 Cơ sở lý thuyết và công nghệ liên quan	5
2.1 Bài toán khai thác công việc từ audio cuộc họp	5
2.2 Speech-to-text và speaker diarization	6
2.3 LLM cho trích xuất thông tin có cấu trúc	7
2.4 RAG, caching và tối ưu suy luận	7
2.5 Guardrails và rủi ro hallucination	7
2.6 LLMOps cho hệ thống NLP vận hành	8
3 Phương pháp	9
3.1 Tổng quan phương pháp	9
3.2 Thiết kế dữ liệu	10
3.3 Xử lý audio và transcript	10
3.4 Trích xuất action items bằng LLM	11
3.5 Chuẩn hoá assignee	11
3.6 Guardrails	12
3.7 Feedback loop và lifecycle LLMOps	12
3.8 Phương pháp đánh giá	12
4 Triển khai hệ thống	14

4.1	Kiến trúc triển khai	14
4.2	Luồng xử lý meeting	15
4.3	Mô hình dữ liệu	16
4.4	API và giao diện người dùng	17
4.5	Xác thực, phân quyền và xử lý dữ liệu nhạy cảm	18
4.6	Monitoring và observability	18
4.6.1	Prometheus và Grafana	19
4.6.2	LangSmith trace	19
4.6.3	Anomaly detection	20
4.7	MLOps và vòng cải tiến	20
4.8	Giới hạn triển khai	20
5	Đánh giá	22
5.1	Thiết lập đánh giá	22
5.2	Phương pháp tính metric	23
5.3	Kết quả định lượng	24
5.4	Đánh giá độ trễ	25
5.5	Kiểm thử chức năng và vận hành	25
5.6	Phân tích kết quả	26
5.7	Giới hạn của đánh giá	26
6	Thách thức và hạn chế	27
6.1	Chất lượng audio và lỗi lan truyền trong pipeline	27
6.2	Diarization và định danh người nói	27
6.3	Assignee resolution	28
6.4	Giới hạn của dữ liệu đánh giá	28
6.5	Hiệu năng và tài nguyên tính toán	28
6.6	Bảo mật và dữ liệu nhạy cảm	29
6.7	Mức độ sẵn sàng triển khai	29
7	Hướng phát triển	30
7.1	Mở rộng dữ liệu đánh giá và feedback	30
7.2	Đánh giá audio end-to-end	30
7.3	Cải thiện assignee và deadline extraction	31
7.4	Nâng cấp evaluation harness	31
7.5	Fine-tuning và model lifecycle	32
7.6	Nâng cao vận hành	32
7.7	Hỗ trợ tiếng Việt và đa ngôn ngữ	32
8	Kết luận	33

Tài liệu tham khảo	34
A Cấu trúc dự án hiện tại	36
B Hướng dẫn chạy	37
B.1 Chạy môi trường mặc định	37
B.2 Chạy profile MLOps	37
B.3 Deploy model promoted	37
C Ví dụ API	39
C.1 Upload meeting	39
C.2 Lấy trạng thái meeting	39
C.3 Gửi feedback	39
C.4 Xuất feedback cho fine-tuning	40
D Cấu hình môi trường quan trọng	41

List of Figures

3.1 Pipeline phương pháp từ audio cuộc họp đến dữ liệu hiệu chỉnh	9
4.1 Kiến trúc runtime của Meeting AI Agent	14
4.2 Mô hình dữ liệu chính. Các bảng nghiệp vụ chính tham chiếu trực tiếp về <code>meetings</code> ; các liên kết logic khác được mô tả trong phần nội dung. .	16
5.1 Các metric chất lượng chính của baseline	24

List of Tables

2.1 Vai trò của các nhóm kỹ thuật trong Meeting AI Agent	6
3.1 Các loại dữ liệu chính trong phương pháp	10
4.1 Vai trò của các thành phần triển khai	15
4.2 Các bảng dữ liệu chính	17

4.3	Các API chính của hệ thống	18
4.4	Metrics vận hành dùng cho Prometheus/Grafana	19
5.1	Cấu hình benchmark baseline	22
5.2	Kết quả baseline trên benchmark 100 mẫu	24

Tóm tắt

Báo cáo trình bày **Meeting AI Agent**, một hệ thống xử lý audio cuộc họp theo định hướng LLMOps. Hệ thống nhận file âm thanh, tạo transcript theo lượt nói, sinh tóm tắt, trích xuất action items có cấu trúc và cho phép người dùng hiệu chỉnh kết quả. Mục tiêu chính là chuyển nội dung cuộc họp từ dữ liệu không có cấu trúc thành thông tin có thể truy vết, đánh giá và tích hợp vào quy trình vận hành.

Kiến trúc hiện tại gồm Next.js frontend, FastAPI backend, Celery worker, Redis, PostgreSQL, Ollama/Qwen2.5-3B, Prometheus và Grafana. Audio được chuẩn hoá, đưa qua WhisperX để nhận dạng tiếng nói tự động (ASR), sau đó tổ chức thành transcript turns và truyền cho LLM để tạo summary hoặc action items dạng JSON. Trước khi lưu kết quả, hệ thống áp dụng guardrails như kiểm tra schema, lọc prompt injection, kiểm tra ngày hạn, phát hiện dấu hiệu hallucination và che một số thông tin nhạy cảm bằng regex-based PII masking.

LLMOps được thể hiện qua cơ chế feedback, lưu vết dữ liệu, benchmark, monitoring và quy trình chuẩn bị cho fine-tuning. Khi người dùng sửa task description, assignee, due date, đánh dấu false positive hoặc bổ sung task bị bỏ sót, các hiệu chỉnh này được lưu trong PostgreSQL và có thể export phục vụ đánh giá hoặc huấn luyện sau này. Fine-tuning hiện được xem là luồng đã chuẩn bị về kỹ thuật, chưa phải bằng chứng chính cho chất lượng baseline.

Đánh giá baseline được thực hiện trên 100 mẫu từ tập tham chiếu synthetic, gồm transcript cuộc họp tổng hợp và action items kỳ vọng. Với Qwen2.5-3B ở chế độ few-shot, hệ thống đạt precision 0.8604, recall 0.6665, F1 0.6886, assignee accuracy 0.5232, hallucination rate 0.0% và schema failure rate 0.0%. Kết quả cho thấy pipeline ổn định về định dạng và kiểm soát hallucination trên benchmark tổng hợp, trong khi recall và assignee accuracy vẫn là các hướng cần cải thiện.

Chapter 1

Giới thiệu

1.1 Bối cảnh và động lực

Trong doanh nghiệp, cuộc họp thường tạo ra nhiều quyết định, công việc cần theo dõi và cam kết giữa các thành viên. Tuy nhiên, các thông tin này thường nằm rải rác trong audio, ghi chú cá nhân hoặc tin nhắn sau họp. Điều này làm action items dễ bị bỏ sót, deadline không rõ ràng và người phụ trách có thể bị gán sai.

Meeting AI Agent được xây dựng để chuyển audio cuộc họp thành dữ liệu có cấu trúc: transcript theo lượt nói, tóm tắt, danh sách action items, assignee, due date, priority và bằng chứng transcript liên quan. Trọng tâm của dự án không chỉ là sinh văn bản bằng LLM, mà là xây dựng một pipeline có thể vận hành, đo lường và cải tiến theo hướng LLMOps.

Trong báo cáo này, LLMOps được hiểu là tập hợp các thực hành để quản lý vòng đời của hệ thống dùng mô hình ngôn ngữ lớn: dữ liệu, prompt, suy luận, đánh giá, monitoring, feedback và kiểm soát thay đổi mô hình.

1.2 Bài toán NLP

Dự án kết hợp ba bài toán chính.

1.2.1 Nhận dạng tiếng nói tự động

Nhận dạng tiếng nói tự động (Automatic Speech Recognition – ASR) chuyển audio thành văn bản. Trong hệ thống này, WhisperX nhận audio đã được chuẩn hoá và trả về transcript kèm thông tin thời gian. WhisperX được chọn vì hỗ trợ alignment tốt hơn ở mức từ hoặc đoạn nói, giúp truy vết action items về đoạn hội thoại gốc. Chất lượng ASR vẫn phụ thuộc vào tiếng ồn, accent, ngôn ngữ và tài nguyên tính toán.

1.2.2 Phân tách người nói

Speaker diarization xác định “ai nói khi nào”. Khác với ASR, diarization không tạo nội dung văn bản mà gán các đoạn audio cho speaker labels như `SPEAKER_00` hoặc

SPEAKER_01. Thông tin này giúp hệ thống hiểu ngữ cảnh người nói và hỗ trợ gán assignee. Giới hạn chính là audio nhiều, giọng nói chồng lấn hoặc các speaker có giọng tương tự nhau.

1.2.3 Trích xuất thông tin có cấu trúc

Sau khi có transcript, hệ thống dùng Qwen2.5-3B qua Ollama để trích xuất action items. Đầu vào của LLM là prompt chứa ngày họp, danh sách người tham gia, transcript chunk và hướng dẫn xuất JSON. Đầu ra là danh sách task có mô tả, assignee, due date, priority và source turns. Vì backend cần dữ liệu có cấu trúc, kết quả LLM được kiểm tra bằng schema và guardrails trước khi lưu.

1.3 Mục tiêu hệ thống

Các mục tiêu chính của dự án gồm:

1. Xử lý audio cuộc họp end-to-end qua giao diện web.
2. Tạo transcript theo speaker turns để phục vụ truy vết.
3. Trích xuất summary và action items bằng LLM chạy local.
4. Chuẩn hoá assignee theo worker roster và cho phép người dùng hiệu chỉnh.
5. Lưu meeting, transcript, tasks, feedback corrections và calendar events vào PostgreSQL.
6. Tích hợp feedback loop cho đánh giá và fine-tuning trong tương lai.
7. Theo dõi hệ thống bằng Prometheus/Grafana.
8. Chuẩn bị benchmark, dataset validation, retrain trigger và promotion gate theo hướng LLMOps.

1.4 Phạm vi dự án

Phiên bản hiện tại là hệ thống local-first chạy bằng Docker Compose. Runtime stack gồm Next.js, FastAPI, Celery, Redis, PostgreSQL, Ollama, Prometheus và Grafana. Cách triển khai này phù hợp với mục tiêu môn học: chứng minh một pipeline NLP/LLMOps hoàn chỉnh mà không phụ thuộc vào dịch vụ LLM trả phí.

Dự án đã tập trung vào các năng lực cốt lõi: xử lý audio, tạo transcript, trích xuất action items, lưu trữ có truy vết, human review, Google Calendar sync, benchmark

baseline và monitoring. Một số phần như fine-tuning GPU, triển khai qua domain public, autoscaling, tuân thủ dữ liệu đầy đủ và quản trị multi-tenant được xem là hướng mở rộng.

1.5 Đóng góp chính

Dự án đóng góp các thành phần sau:

- Pipeline audio-to-action-items kết hợp ASR, diarization, LLM extraction, guardrails và worker assignment.
- Backend bất đồng bộ bằng FastAPI, Celery, Redis và PostgreSQL cho các job audio dài.
- Web UI cho upload, theo dõi tiến trình, review task, sửa kết quả và đồng bộ lịch.
- Feedback loop lưu dữ liệu hiệu chỉnh của người dùng để phục vụ đánh giá và cải tiến mô hình.
- Monitoring và benchmark baseline để đo chất lượng extraction, schema stability, hallucination rate và các chỉ số vận hành.

1.6 Cấu trúc báo cáo

Chương 2 trình bày cơ sở lý thuyết và công nghệ liên quan. Chương 3 mô tả phương pháp luận, bao gồm pipeline dữ liệu, lựa chọn mô hình, prompt engineering, guardrails và đánh giá. Chương 4 trình bày kiến trúc triển khai. Chương 5 phân tích thiết lập đánh giá và kết quả baseline. Chương 6 thảo luận thách thức và giới hạn. Chương 7 nêu hướng phát triển. Chương 8 tổng kết dự án.

Chapter 2

Cơ sở lý thuyết và công nghệ liên quan

Chương này trình bày các nền tảng kỹ thuật liên quan đến Meeting AI Agent. Mục tiêu không phải liệt kê công nghệ, mà làm rõ vì sao từng nhóm kỹ thuật cần thiết cho bài toán chuyển audio cuộc họp thành action items có thể kiểm chứng.

2.1 Bài toán khai thác công việc từ audio cuộc họp

Trong môi trường doanh nghiệp, audio cuộc họp thường chứa nhiều thông tin cần theo dõi sau cuộc họp: quyết định, công việc được giao, người phụ trách và thời hạn. Các báo cáo về môi trường làm việc hiện đại cho thấy meeting và trao đổi số chiếm tỷ trọng lớn trong thời gian làm việc tri thức [1]. Vì vậy, giá trị của hệ thống không chỉ nằm ở việc lưu lại transcript, mà ở khả năng biến hội thoại thành thông tin có thể hành động: ai cần làm gì, trước thời hạn nào và dựa trên phát biểu nào.

So với các bài toán NLP văn bản thuần túy, audio cuộc họp có thêm nhiều nguồn nhiễu: chất lượng âm thanh, nhiều người nói, phát biểu ngắt quãng, đại từ thay cho tên người và deadline được nói theo ngữ cảnh. Do đó, bài toán audio-to-action-items cần một chuỗi xử lý gồm ASR, diarization, LLM extraction và guardrails, thay vì chỉ dùng một bước tóm tắt văn bản.

Table 2.1: Vai trò của các nhóm kỹ thuật trong Meeting AI Agent

Nhóm kỹ thuật	Vai trò chính	Rủi ro cần kiểm soát
ASR	Chuyển audio thành transcript	Nhiều âm thanh, accent, lỗi nhận dạng từ
Diarization	Xác định lượt nói theo speaker	Nói chồng, speaker có giọng tương tự
LLM extraction	Trích xuất summary và action items	Hallucination, sai schema, bỏ sót task
RAG/cache	Bổ sung ngữ cảnh và giảm chi phí suy luận	Truy hồi sai ngữ cảnh, cache không phù hợp
Guardrails	Kiểm tra input/output của model	Không loại bỏ hoàn toàn rủi ro nội dung
LLMOps	Đánh giá, monitoring, feedback và kiểm soát thay đổi	Metric không đại diện dữ liệu thật, regression

2.2 Speech-to-text và speaker diarization

Nhận dạng tiếng nói tự động (ASR) là bước chuyển audio thành transcript. Whisper là một hướng tiếp cận mạnh mẽ cho ASR nhờ được huấn luyện trên dữ liệu âm thanh quy mô lớn và có khả năng khái quát tốt trên nhiều điều kiện âm thanh [7]. WhisperX mở rộng hướng này bằng cơ chế alignment để tạo timestamp chính xác hơn, đặc biệt hữu ích với audio dài [2].

Timestamp không chỉ phục vụ hiển thị transcript. Trong hệ thống meeting, timestamp và speaker turn giúp liên kết một action item với bằng chứng hội thoại nguồn. Đây là nền tảng cho khả năng audit: người dùng có thể kiểm tra vì sao một task được tạo ra thay vì chỉ nhìn thấy kết quả cuối.

Speaker diarization trả lời câu hỏi “ai nói khi nào”. Pyannote là một toolkit phổ biến cho diarization và cung cấp pipeline nhận diện đoạn nói theo speaker [3]. Diarization bổ sung cho ASR: ASR sinh nội dung văn bản, còn diarization tổ chức nội dung đó theo người nói. Giới hạn chung của nhóm kỹ thuật này là nhạy với tiếng ồn, overlapping speech, accent và chất lượng microphone; vì vậy kết quả diarization cần được người dùng kiểm tra trong các trường hợp quan trọng.

2.3 LLM cho trích xuất thông tin có cấu trúc

Các mô hình ngôn ngữ lớn dựa trên Transformer [12] có khả năng học ngữ cảnh và làm theo chỉ dẫn ở nhiều tác vụ NLP. Nghiên cứu về few-shot learning cho thấy LLM có thể thực hiện tác vụ mới khi được cung cấp hướng dẫn và ví dụ phù hợp [11]. Trong bài toán này, LLM không chỉ tóm tắt nội dung mà còn phải trích xuất action items theo schema để backend có thể xử lý tiếp.

Qwen2.5 được chọn làm model phục vụ suy luận cục bộ vì cân bằng giữa khả năng instruction-following và chi phí vận hành [4]. Việc chạy model qua Ollama giúp hệ thống không phụ thuộc vào API thương mại và phù hợp với bối cảnh demo local. Đối lại, model kích thước 3B, đặc biệt khi chạy trên CPU, có giới hạn về tốc độ và năng lực suy luận so với các model lớn hơn.

Structured extraction yêu cầu đầu ra ổn định hơn sinh văn bản tự do. Prompt cần chỉ rõ schema mong muốn, còn backend phải kiểm tra lại JSON trước khi lưu. Cách tiếp cận này giúp kết quả LLM có thể đi tiếp vào database, giao diện review, calendar sync và evaluation pipeline mà không phụ thuộc vào diễn giải thủ công.

2.4 RAG, caching và tối ưu suy luận

Retrieval-Augmented Generation (RAG) bổ sung ngữ cảnh liên quan cho LLM thông qua bước truy hồi. Trong hệ thống này, văn bản transcript hoặc thông tin speaker có thể được biểu diễn dưới dạng embedding, sau đó FAISS hỗ trợ tìm các đoạn gần nhất trong không gian vector [8]. Vai trò của RAG là giảm mất ngữ cảnh khi transcript dài hoặc khi một chunk cần tham chiếu thông tin đã xuất hiện trước đó.

Caching là một kỹ thuật tối ưu vận hành khác. Redis không phải mô hình AI, nhưng có thể lưu kết quả cho các prompt đã xử lý để giảm số lần gọi LLM lặp lại. Trong môi trường local, nơi LLM chạy chậm hơn cloud GPU, caching giúp giảm latency và tài nguyên tính toán cho các request trùng hoặc gần trùng.

2.5 Guardrails và rủi ro hallucination

LLM có thể sinh câu trả lời đúng định dạng nhưng sai nội dung, hoặc sinh thêm thông tin không có trong input. Hiện tượng này thường được gọi là hallucination và là rủi ro quan trọng trong các hệ thống sinh ngôn ngữ [10]. Với meeting action items, hallucination có thể tạo ra task không tồn tại, gán sai người phụ trách hoặc suy diễn deadline không được nói trong cuộc họp.

Guardrails là lớp kiểm soát quanh input và output của model. Trong dự án này, guardrails bao gồm kiểm tra JSON schema, lọc dấu hiệu prompt injection, regex-based PII masking, kiểm tra ngày hạn bất thường và kiểm tra một số dấu hiệu hallucination. Các cơ chế này không loại bỏ hoàn toàn rủi ro, nhưng giúp giảm lỗi phổ biến và làm kết quả phù hợp hơn với yêu cầu lưu trữ có cấu trúc.

2.6 LLMOps cho hệ thống NLP vận hành

MLOps/LLMOps nhấn mạnh việc đưa mô hình vào vòng đời có dữ liệu, đánh giá, triển khai, monitoring và kiểm soát thay đổi [9]. Với LLM, vòng đời này phức tạp hơn vì chất lượng phụ thuộc không chỉ vào trọng số model mà còn vào prompt, dữ liệu ngữ cảnh, guardrails, feedback và cách đánh giá regression.

Từ góc nhìn vận hành, điều này dẫn tới một nguyên tắc quan trọng: mọi thay đổi về dữ liệu, prompt hoặc model cần được đo bằng cùng một bộ benchmark trước khi đưa vào sử dụng. Đây là cơ sở cho các thành phần như feedback export, regression evaluation, monitoring và promotion gate trong Meeting AI Agent.

Chapter 3

Phương pháp

Chương này mô tả phương pháp thiết kế hệ thống ở mức pipeline và dữ liệu. Các chi tiết triển khai cụ thể như API, database schema và Docker services được trình bày ở Chương 4.

3.1 Tổng quan phương pháp

Meeting AI Agent được thiết kế như một pipeline chuyển đổi từ audio sang dữ liệu có cấu trúc. Đầu vào là file audio cuộc họp. Đầu ra gồm tóm tắt cuộc họp và danh sách action items có các trường chính: mô tả công việc, người phụ trách, thời hạn, mức ưu tiên và lượt transcript nguồn.

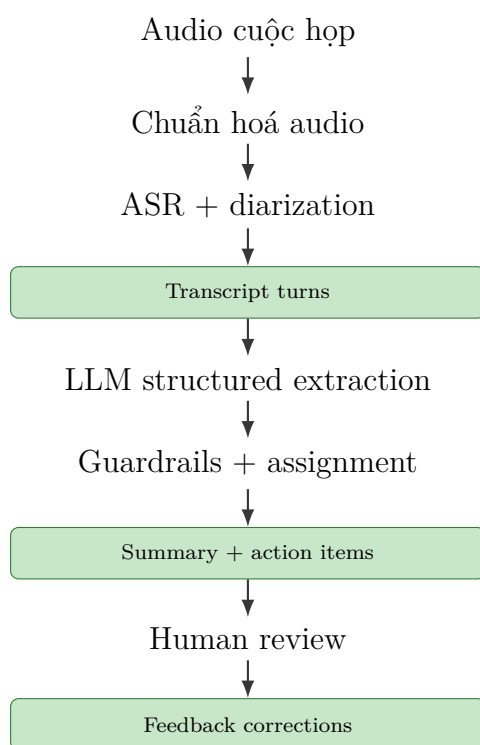


Figure 3.1: Pipeline phương pháp từ audio cuộc họp đến dữ liệu hiệu chỉnh

Thiết kế này tách rõ ba lớp trách nhiệm. Lớp xử lý tiếng nói tạo transcript có speaker turns. Lớp LLM chuyển transcript thành dữ liệu có cấu trúc. Lớp LLMOps lưu vết,

đánh giá, nhận feedback và chuẩn bị dữ liệu cho các vòng cải tiến.

3.2 Thiết kế dữ liệu

Hệ thống sử dụng nhiều loại dữ liệu khác nhau, mỗi loại phục vụ một mục đích riêng. Bảng 3.1 tóm tắt vai trò của các loại dữ liệu chính.

Table 3.1: Các loại dữ liệu chính trong phương pháp

Loại dữ liệu	Nội dung	Vai trò
Audio đầu vào	File mp3, wav, m4a hoặc định dạng tương đương	Nguồn dữ liệu ban đầu của pipeline
Transcript turns	Các lượt nói gồm speaker, thời gian và nội dung	Làm input cho LLM và bằng chứng truy vết
Worker roster	Danh sách nhân sự, tên, email, alias và vai trò	Chuẩn hoá assignee và giảm lỗi gán người
Action items	Task có description, assignee, due date, priority và source turns	Kết quả vận hành chính của hệ thống
Feedback corrections	Dữ liệu người dùng hiệu chỉnh sau review	Nguồn cải tiến, đánh giá và fine-tuning sau này
Tập đánh giá tham chiếu	Transcript tổng hợp kèm action items kỳ vọng	Benchmark baseline và so sánh candidate model

Dữ liệu tổng hợp (synthetic data) được dùng để bootstrap training hoặc evaluation trong điều kiện kiểm soát. Ví dụ, `data/eval/gold_synthetic_205.jsonl` là tập đánh giá tham chiếu gồm transcript cuộc họp tổng hợp và action items kỳ vọng. Vì tập này không thay thế dữ liệu hợp thật đã được kiểm chứng bởi con người, báo cáo chỉ dùng nó như benchmark có kiểm soát cho baseline.

Dữ liệu hiệu chỉnh của người dùng (feedback corrections) là các bản ghi sinh ra khi người dùng sửa kết quả model, chẳng hạn sửa mô tả task, assignee, due date, đánh dấu false positive hoặc bổ sung task bị bỏ sót. Đây là nguồn dữ liệu quan trọng nhất cho vòng cải tiến vì nó phản ánh lỗi thật trong quá trình sử dụng.

3.3 Xử lý audio và transcript

Audio đầu vào được chuẩn hoá trước khi đưa vào mô hình nhận dạng tiếng nói. Mục tiêu của bước này là giảm khác biệt giữa các định dạng file và tạo đầu vào nhất quán

cho ASR. Sau đó, WhisperX tạo transcript và thông tin thời gian; diarization gán các đoạn nói cho speaker labels như `SPEAKER_00`, `SPEAKER_01`.

Đầu ra chuẩn hoá của bước này là danh sách `TranscriptTurn`. Mỗi turn biểu diễn một đoạn nói gồm speaker, thời gian bắt đầu, thời gian kết thúc và nội dung văn bản. Đây là đơn vị trung tâm của hệ thống: prompt LLM được dựng từ transcript turns, action item lưu `source_turn_ids`, còn UI dùng các turn này để hiển thị bằng chứng khi người dùng review.

3.4 Trích xuất action items bằng LLM

LLM được dùng cho hai nhiệm vụ: tóm tắt cuộc họp và trích xuất action items. Với action item extraction, hệ thống xây dựng prompt từ bốn thành phần: ngày họp, danh sách người tham gia, transcript chunk và hướng dẫn xuất JSON theo schema. Việc chia transcript thành chunk giúp kiểm soát độ dài prompt khi cuộc họp dài.

Đầu ra mong muốn không phải văn bản tự do mà là JSON array. Mỗi phần tử biểu diễn một task với các trường chính như `description`, `assignee`, `due_date`, `priority` và `source_turn_ids`. Cách tiếp cận này giúp kết quả LLM có thể được lưu, kiểm tra và đánh giá bằng chương trình.

Sau khi model trả kết quả, hệ thống không lưu trực tiếp raw output. Kết quả phải được parse, kiểm tra schema và chuẩn hoá assignee. Nếu output không hợp lệ, task có thể bị loại hoặc đưa vào nhóm cần review thay vì được xem là kết quả đã xác nhận.

3.5 Chuẩn hoá assignee

Trong transcript, người phụ trách có thể được nhắc bằng tên đầy đủ, tên ngắn, alias hoặc chỉ xuất hiện gián tiếp qua lượt nói. Vì vậy, hệ thống sử dụng worker roster để chuẩn hoá assignee. Về phương pháp, assignee từ LLM được so khớp với roster bằng tên và alias; nếu không đủ tin cậy, task được đưa vào trạng thái unresolved hoặc human review.

Thiết kế này tránh việc backend lưu các tên không nhất quán như “Bob”, “Bobby” và “Robert” như ba người khác nhau. Nó cũng phản ánh thực tế rằng một phần lỗi gán người không nên được tự động quyết định nếu transcript hoặc diarization chưa đủ rõ.

3.6 Guardrails

Guardrails được đặt quanh cả input và output của LLM. Ở phía input, hệ thống kiểm tra một số mẫu prompt injection hoặc jailbreak và che các dạng thông tin nhạy cảm phổ biến như email, số điện thoại, SSN, thẻ, IP và URL bằng regex-based PII masking. Cơ chế này giảm rủi ro đưa thông tin nhạy cảm hoặc chỉ dẫn độc hại vào prompt.

Ở phía output, hệ thống kiểm tra JSON schema, chuẩn hoá ngày hạn và phát hiện một số dấu hiệu hallucination. Hallucination trong ngữ cảnh này là trường hợp model tạo task không có đủ bằng chứng trong transcript. Guardrails không đảm bảo loại bỏ mọi lỗi, nhưng giúp biến output của LLM thành dữ liệu có thể kiểm soát trước khi đi vào database.

3.7 Feedback loop và lifecycle LLMOps

Sau khi hệ thống sinh task, người dùng có thể review và gửi dữ liệu hiệu chỉnh. Một correction có thể sửa task description, assignee, due date, đánh dấu task là false positive hoặc bổ sung task bị thiếu. Các correction được lưu trong database và có thể export thành JSONL.

Luồng LLMOps của dự án gồm bốn bước chính: thu thập feedback, validate dữ liệu, đánh giá regression và chuẩn bị fine-tuning khi đủ điều kiện. Fine-tuning dùng LoRA/QLoRA [6, 5] là hướng đã chuẩn bị về kỹ thuật, nhưng trong báo cáo này không được xem là bằng chứng chính cho chất lượng hiện tại nếu chưa có artifact huấn luyện và kết quả so sánh cụ thể.

3.8 Phương pháp đánh giá

Đánh giá chất lượng action item extraction cần trả lời hai câu hỏi: model có tìm đúng task hay không và các trường quan trọng như assignee có đúng không. Trong benchmark hiện tại, mỗi mẫu gồm transcript turns, roster, ngày họp và danh sách action items tham chiếu.

Quy trình so sánh hoạt động như sau. Hệ thống chạy pipeline trích xuất trên transcript để tạo danh sách predicted tasks. Mỗi predicted task được so với các reference tasks chủ yếu bằng trường **description**. Matcher dùng BM25 trước, tức một phương pháp xếp hạng độ liên quan văn bản dựa trên mức độ trùng khớp từ khóa có trọng số; nếu không đạt ngưỡng, nó fallback sang Jaccard token overlap. Mỗi reference task chỉ được match một lần.

Từ quy trình trên, các đại lượng được định nghĩa như sau:

- **True positive:** predicted task match được một reference task.
- **False positive:** predicted task không match được reference task nào.
- **False negative:** reference task không được predicted task nào match.
- **Precision:** tỷ lệ task dự đoán là đúng trên tổng số task dự đoán.
- **Recall:** tỷ lệ task tham chiếu được hệ thống tìm thấy.
- **F1:** trung bình điều hoà giữa precision và recall.
- **Assignee accuracy:** tỷ lệ assignee đúng trên các task đã match description.
- **Schema failure rate:** tỷ lệ mẫu mà output không parse hoặc validate được theo schema.
- **Hallucination rate:** tỷ lệ task dự đoán thiếu bằng chứng rõ ràng trong transcript.

Precision quan trọng vì task sai có thể làm người dùng mất niềm tin và tạo công việc không tồn tại. Recall cũng quan trọng vì task bị bỏ sót làm giảm giá trị tự động hoá. Trong baseline hiện tại, benchmark synthetic giúp kiểm soát regression và so sánh candidate model, nhưng chưa thay thế đánh giá trên meeting thật với audio nhiều, cách nói tự nhiên và tổ chức nhân sự thực tế.

Chapter 4

Triển khai hệ thống

Chương này mô tả cách hiện thực Meeting AI Agent thành một hệ thống chạy được bằng Docker Compose. Trọng tâm của chương không phải là lặp lại thuật toán ở Chương 3, mà là làm rõ các quyết định kỹ thuật: thành phần runtime, luồng xử lý bất đồng bộ, mô hình dữ liệu, API, cơ chế quan sát hệ thống và các điểm hỗ trợ LLMOps.

4.1 Kiến trúc triển khai

Hệ thống được triển khai theo kiến trúc nhiều dịch vụ. Frontend xử lý tương tác người dùng; backend cung cấp API và ghi nhận trạng thái; worker thực thi pipeline audio-NLP; các dịch vụ nền đảm nhiệm hàng đợi, lưu trữ, suy luận mô hình và monitoring. Kiến trúc này phù hợp với bài toán xử lý audio vì thời gian chạy ASR, diarization và LLM thường dài hơn một request HTTP thông thường.

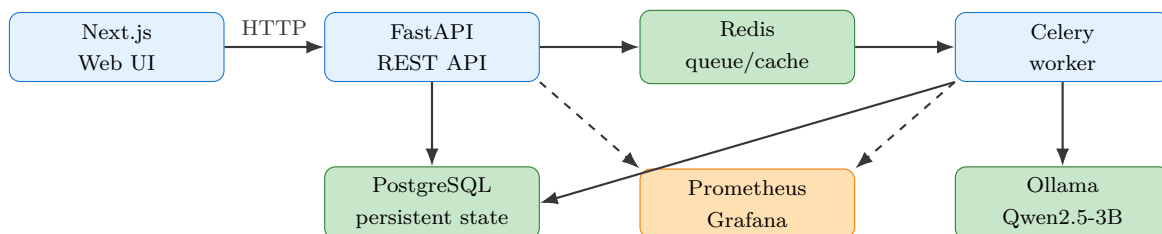


Figure 4.1: Kiến trúc runtime của Meeting AI Agent

Thành phần	Trách nhiệm chính	Dữ liệu vào/ra
Next.js Web UI	Upload audio, hiển thị trạng thái xử lý, review transcript, speaker mapping và task feedback.	Nhận audio và thao tác người dùng; gọi API nội bộ.
FastAPI backend	Tạo meeting job, quản lý dữ liệu, phân quyền theo người dùng, cung cấp endpoint cho frontend và monitoring.	Nhận HTTP request; ghi PostgreSQL; đẩy job vào Redis.
Celery worker	Chạy pipeline ASR, diarization, prompt construction, LLM extraction, guardrails và lưu kết quả.	Nhận job từ Redis; đọc/ghi artifact và PostgreSQL.
PostgreSQL	Lưu trạng thái hệ thống có cấu trúc.	Meetings, transcript turns, tasks, feedback, workers, calendar events.
Redis	Làm Celery broker/result backend và cache cho một số lời gọi suy luận.	Queue message, trạng thái job ngắn hạn, cache prompt.
Ollama	Phục vụ LLM cục bộ. Trong bản demo dùng Qwen2.5-3B.	Nhận prompt; trả summary hoặc JSON action items.
Prometheus/Grafana	Thu thập và hiển thị metrics vận hành.	Scrape /metrics; trực quan hóa latency, job, token, guardrail.

Table 4.1: Vai trò của các thành phần triển khai

Với Docker Compose, các dịch vụ cốt lõi gồm `postgres`, `redis`, `ollama`, `migrator`, `api`, `worker`, `web`, `prometheus` và `grafana`. Profile `mlops` bổ sung `beat`, `trainer` và `mlflow` cho các tác vụ đánh giá/huấn luyện mở rộng.

4.2 Luồng xử lý meeting

Một meeting được xử lý bất đồng bộ để tránh việc HTTP request bị khóa trong suốt quá trình audio processing. Luồng chính gồm năm bước:

1. Người dùng upload audio từ Web UI. Frontend gửi request `POST /meetings` đến backend.
2. Backend tạo bản ghi `meetings` với trạng thái `pending`, lưu metadata ban đầu và đẩy job vào Redis.
3. Celery worker nhận job, chuyển trạng thái sang `processing`, sau đó chạy pipeline ASR, diarization, chuẩn hóa transcript và gọi LLM.
4. Kết quả được kiểm tra bằng guardrails, ghi vào các bảng `transcript_turns`, `tasks`, `meeting_participants` và cập nhật `meetings`.
5. Frontend polling `GET /meetings/{id}` để hiển thị trạng thái. Khi hoàn tất, người dùng review transcript/task và gửi feedback nếu cần.

Trạng thái meeting được lưu trong cơ sở dữ liệu thay vì chỉ nằm trong bộ nhớ worker. Nhờ vậy, nếu frontend refresh hoặc worker xử lý lâu, người dùng vẫn có thể tiếp tục

theo dõi cùng một meeting. Với các job bị treo ở trạng thái **pending**, backend có logic đánh dấu lỗi sau một khoảng thời gian nhất định để tránh hiển thị trạng thái không kết thúc.

4.3 Mô hình dữ liệu

Thực thể trung tâm của hệ thống là **Meeting**. Mỗi audio upload sinh ra một meeting, và các dữ liệu dẫn xuất đều tham chiếu về **meeting_id**. Thiết kế này giúp truy vết một action item về transcript gốc, feedback của người dùng, artifact xử lý và phiên bản mô hình đã dùng.

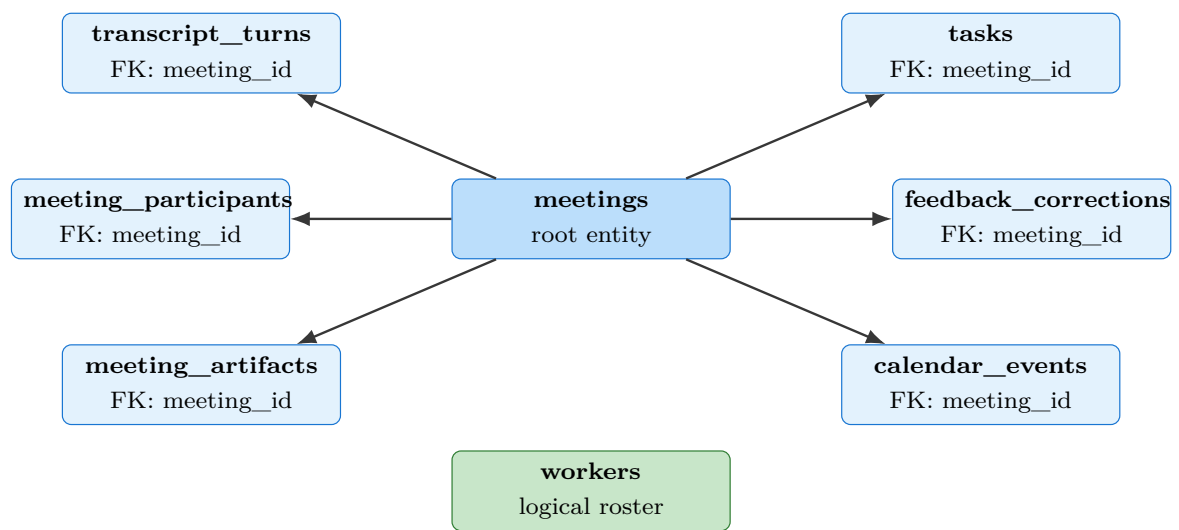


Figure 4.2: Mô hình dữ liệu chính. Các bảng nghiệp vụ chính tham chiếu trực tiếp về **meetings**; các liên kết logic khác được mô tả trong phần nội dung.

Bảng	Vai trò	Trường đáng chú ý
meetings	Bản ghi gốc của một lần xử lý audio.	id, status, owner_user_id, audio_filename, duration_ms, participants, summary_text, run_metrics, transcript_turns, error, model_version.
transcript_turns	Lưu transcript chuẩn hoá theo lượt nói. Đây là bảng phục vụ truy vấn/UI, trong khi meetings.transcript_turns là snapshot JSONB.	meeting_id, turn_id, speaker_id, speaker_name, worker_id, start_ms, end_ms, text, asr_confidence.
tasks	Lưu action items sau LLM extraction và guardrails.	meeting_id, task_id, description, assignee, assignee_id, due_date, priority, status, extraction_confidence, source_turn_ids, bucket.
feedback_corrections	Lưu correction, false positive hoặc missing task do người dùng gửi.	meeting_id, task_id, reviewer, các cặp original_*/corrected_*, is_false_positive, is_missing, submitted_at.
meeting_participants	Ánh xạ speaker ẩn danh sang tên người tham gia trong một meeting.	meeting_id, speaker_id, display_name, worker_id, email.
workers	Danh bạ người tham gia/người có thể được gán task, được scope theo user.	worker_id, owner_user_id, name, email, role, aliases, skills.
meeting_artifacts	Lưu artifact thô hoặc dẫn xuất để audit.	meeting_id, artifact_type, storage_uri, payload, checksum, artifact_metadata.
calendar_events	Theo dõi việc đồng bộ task sang Google Calendar.	meeting_id, task_id, user_id, provider, provider_event_id, html_link, status, last_error.

Table 4.2: Các bảng dữ liệu chính

Hai điểm quan trọng của mô hình dữ liệu là `owner_user_id` và `source_turn_ids`. Trường `owner_user_id` giúp tách dữ liệu theo người dùng khi bật xác thực. Trường `source_turn_ids` giúp mỗi task giữ liên kết logic đến các lượt nói đã tạo ra task đó; đây là cơ sở để UI hiển thị evidence, người dùng sửa lỗi có ngữ cảnh, và bộ đánh giá so sánh dự đoán với dữ liệu tham chiếu. Trong schema hiện tại, `calendar_events.task_id` và `meeting_participants.worker_id` là mã tham chiếu logic, không phải foreign key vật lý đến `tasks` hoặc `workers`.

4.4 API và giao diện người dùng

Backend cung cấp API REST cho ba nhóm chức năng: xử lý meeting, quản lý danh bạ người tham gia, và vận hành hệ thống. Bảng 4.3 liệt kê các endpoint chính.

Endpoint	Mục đích	Nhóm
POST /meetings	Upload audio và tạo job xử lý bất đồng bộ.	Meeting
GET /meetings/{id}	Lấy trạng thái, transcript, summary và tasks của một meeting.	Meeting
GET /meetings	Liệt kê lịch sử meeting theo scope người dùng.	Meeting
DELETE /meetings/{id}	Xóa dữ liệu meeting và các bản ghi liên quan.	Meeting
POST /meetings/{id}/feedback	Gửi correction, false positive hoặc missing task.	Feedback
POST /meetings/{id}/participants/{speaker}/resolve	Gán speaker diarization vào một worker cụ thể.	Review
GET/POST/PUT/DELETE /workers	Quản lý roster người tham gia.	Roster
GET /metrics	Xuất Prometheus metrics.	Ops
GET /metrics/business	Xuất chỉ số tổng hợp từ database cho dashboard.	Ops
POST /admin/retrain	Kích hoạt kiểm tra hoặc chạy retraining pipeline.	MLOps
GET /admin/retrain/state	Xem trạng thái retraining gần nhất.	MLOps

Table 4.3: Các API chính của hệ thống

Giao diện người dùng được triển khai bằng Next.js. Các view quan trọng gồm upload audio, meeting history, meeting detail, transcript review, speaker mapping, task correction và roster management. Điểm cần nhấn mạnh là UI không chỉ hiển thị kết quả cuối cùng; nó cho phép người dùng kiểm tra bằng chứng của từng task qua transcript turn. Điều này làm giảm tính “hộp đen” của LLM extraction và tạo dữ liệu feedback có giá trị hơn.

4.5 Xác thực, phân quyền và xử lý dữ liệu nhạy cảm

Ở mức frontend, hệ thống hỗ trợ đăng nhập bằng Google thông qua NextAuth. Khi frontend gọi backend, proxy nội bộ truyền định danh người dùng để backend có thể lọc meetings và workers theo `owner_user_id`. Backend cũng có cơ chế token cho user/admin khi bật cấu hình xác thực. Cách triển khai này đủ cho demo nhiều người dùng ở mức ứng dụng, nhưng chưa được trình bày như một cơ chế bảo mật doanh nghiệp hoàn chỉnh.

Đối với dữ liệu nhạy cảm, hệ thống có lớp masking PII dựa trên biểu thức chính quy. Các mẫu như email, số điện thoại, URL, địa chỉ IP và một số định dạng định danh phổ biến được thay bằng placeholder trước khi đưa vào prompt hoặc log debug. Đây là biện pháp giảm rủi ro rò rỉ thông tin trong quá trình gọi LLM và observability; nó không thay thế cho một hệ thống DLP hoặc kiểm toán tuân thủ đầy đủ.

4.6 Monitoring và observability

Hệ thống dùng ba lớp quan sát chính: Prometheus/Grafana cho metrics định lượng, LangSmith cho trace LLM khi được cấu hình, và anomaly detection cho cảnh báo thống

kê đơn giản.

4.6.1 Prometheus và Grafana

Backend cung cấp endpoint `GET /metrics` theo định dạng Prometheus. Prometheus scrape endpoint này định kỳ, còn Grafana dùng dữ liệu đó để hiển thị dashboard. Các metrics chính gồm:

Metric	Ý nghĩa kỹ thuật
<code>meeting_stage_duration_seconds</code>	Histogram đo thời gian từng stage như ASR, LLM hoặc guardrail. Dùng để phát hiện bottleneck.
<code>meeting_llm_tokens_total</code>	Tổng token đã tiêu thụ bởi LLM. Dùng để theo dõi chi phí và độ dài prompt/response.
<code>meeting_llm_calls_total</code>	Số lần gọi LLM, có label theo model. Dùng để kiểm tra tần suất suy luận.
<code>meeting_jobs_total</code>	Số job theo trạng thái hoàn tất/thất bại. Dùng để theo dõi độ ổn định pipeline.
<code>meeting_tasks_extracted_total</code>	Số task trích xuất thành công. Dùng để quan sát sản lượng đầu ra.
<code>meeting_hallucination_flags_total</code>	Số lần guardrail đánh dấu output có dấu hiệu không bám transcript.
<code>meeting_anomaly_events_total</code>	Số sự kiện bất thường do detector thống kê phát hiện.

Table 4.4: Metrics vận hành dùng cho Prometheus/Grafana

Grafana không quyết định chất lượng mô hình; nó giúp quan sát hệ thống khi chạy. Ví dụ, nếu `meeting_stage_duration_seconds` tăng mạnh ở stage LLM, nguyên nhân có thể đến từ prompt quá dài, model phục vụ chậm, hoặc hàng đợi worker bị quá tải. Các kết luận về chất lượng trích xuất như precision, recall và F1 được trình bày ở Chương 5; các dashboard ở chương này chủ yếu phục vụ vận hành.

4.6.2 LangSmith trace

LangSmith là lớp quan sát dành riêng cho lời gọi LLM. Khi các biến môi trường `LANGCHAIN_TRACING_V2`, `LANGCHAIN_API_KEY` và `LANGCHAIN_PROJECT` được cấu hình, orchestrator tự động trace prompt, response, latency và metadata của các lời gọi LLM. Trong ngữ cảnh hệ thống này, LangSmith hữu ích để trả lời các câu hỏi như: prompt nào tạo ra JSON sai schema, response nào bị guardrail loại, hoặc phiên bản prompt nào làm tăng token/latency.

Vì LangSmith là dịch vụ ngoài và có thể lưu prompt/response, việc bật trace phải đi kèm quy tắc bảo vệ dữ liệu. Trong bản hiện tại, hệ thống có masking PII ở tầng prompt/logging; tuy vậy, báo cáo chỉ xem LangSmith là công cụ debug và phân tích, không phải yêu cầu bắt buộc để hệ thống demo chạy.

4.6.3 Anomaly detection

Ngoài dashboard, hệ thống có detector thống kê dựa trên rolling Z-score. Detector giữ cửa sổ giá trị gần nhất cho từng metric như latency LLM, số task trích xuất, số guardrail flag và tổng token. Một giá trị mới bị đánh dấu bất thường khi lệch khỏi trung bình động quá ngưỡng cấu hình. Cơ chế này đơn giản nhưng hữu ích trong demo: nó giúp phát hiện nhanh các phiên xử lý quá chậm, output có quá ít/quá nhiều task, hoặc token tăng bất thường.

4.7 MLOps và vòng cải tiến

Phần MLOps của hệ thống được thiết kế để nối dữ liệu feedback vào quá trình đánh giá và cải tiến mô hình. Khi người dùng sửa task, backend lưu correction vào `feedback_corrections`; endpoint `/feedback/export` có thể xuất dữ liệu này thành JSONL, tức định dạng mỗi dòng là một JSON object, để phục vụ fine-tuning hoặc đánh giá offline.

Trong Docker Compose, profile `mlops` thêm ba thành phần:

- **beat**: chạy lịch kiểm tra điều kiện retraining, ví dụ số lượng correction đã đủ ngưỡng hay chưa.
- **trainer**: worker riêng cho queue `mlops`, tách training khỏi worker xử lý meeting.
- **mlflow**: tracking server để ghi nhận experiment khi chạy training.

Thiết kế này thể hiện vòng đời LLMOps, nhưng cần phân biệt rõ giữa năng lực đã chuẩn bị và kết quả đã hoàn tất. Dự án hiện tại đã có luồng thu thập feedback, export dữ liệu, trigger retrain và tracking experiment; báo cáo không giả định rằng đã có một mô hình fine-tuned được triển khai vào môi trường vận hành chính thức nếu chưa có artifact và kết quả đánh giá tương ứng.

4.8 Giới hạn triển khai

Phiên bản hiện tại được tối ưu cho demo kỹ thuật bằng Docker Compose trên máy cá nhân. Cách chạy này giúp tái lập môi trường nhanh, nhưng chưa tương đương triển khai vận hành chính thức trên hạ tầng cloud. Một số giới hạn chính gồm:

- Khả năng chịu tải phụ thuộc tài nguyên máy chạy Docker, đặc biệt ở các bước ASR và LLM.
- Ollama chạy cục bộ thuận tiện cho demo, nhưng cần sizing phần cứng rõ ràng nếu

phục vụ nhiều người dùng đồng thời.

- Monitoring đã có metrics và dashboard nền tảng, nhưng chưa có quy trình incident response đầy đủ.
- PII masking hiện dựa trên pattern, chưa bao phủ mọi loại dữ liệu nhạy cảm trong hội thoại tự nhiên.
- Triển khai qua domain public chưa được xem là kết quả chính thức của báo cáo; điểm demo tin cậy là môi trường Docker local.

Những giới hạn này không làm thay đổi mục tiêu của dự án: xây dựng một hệ thống NLP end-to-end có thể xử lý audio meeting, trích xuất action items, cho phép người dùng hiệu chỉnh và tạo nền tảng cho LLMOps. Chương tiếp theo sẽ đánh giá hệ thống bằng các thước đo định lượng và phân tích ý nghĩa của kết quả thu được.

Chapter 5

Đánh giá

Chương này trình bày cách đánh giá hệ thống và kết quả baseline hiện tại. Mục tiêu của phần đánh giá là kiểm tra năng lực trích xuất action items từ transcript cuộc họp, độ ổn định của output có cấu trúc, mức độ hallucination và chi phí thời gian khi chạy mô hình cục bộ. Các kết quả dưới đây được hiểu là baseline trước khi có mô hình fine-tuned từ dữ liệu feedback thật.

5.1 Thiết lập đánh giá

Bộ đánh giá chính là `data/eval/gold_synthetic_205.jsonl`. Mỗi dòng trong file là một mẫu meeting ở định dạng JSON, gồm transcript theo lượt nói, ngày họp, roster người tham gia và danh sách action items tham chiếu. Tập này được tạo theo nhiều miền nội dung như engineering, product, sales, marketing và HR để tránh chỉ kiểm tra một kiểu cuộc họp đơn giản.

Trong lần benchmark gần nhất, hệ thống chạy 100 mẫu đầu tiên của tập trên với cấu hình:

Thành phần	Cấu hình đánh giá
Evaluation harness	<code>src/meeting_agent/mlops/evaluate.py</code>
Benchmark runner	<code>scripts/run_benchmark.py</code>
Tập tham chiếu	<code>data/eval/gold_synthetic_205.jsonl</code>
Số mẫu đánh giá	100 mẫu
Model	qwen2.5:3b chạy qua Ollama
Prompt mode	<code>few_shot</code>
Ngày chạy	2026-05-23

Table 5.1: Cấu hình benchmark baseline

Điểm cần làm rõ là benchmark này đánh giá phần action-item extraction từ transcript đã có sẵn, không đo trực tiếp Word Error Rate của ASR hoặc Diarization Error Rate của speaker diarization. Do đó, kết quả phản ánh chất lượng của bước LLM extraction,

guardrails và assignee resolution trên transcript tham chiếu. Đánh giá audio end-to-end cần thêm tập audio có transcript và speaker annotation được kiểm chứng thủ công.

5.2 Phương pháp tính metric

Mỗi mẫu có hai danh sách: action items tham chiếu và action items dự đoán bởi hệ thống. Việc so sánh không dùng so khớp chuỗi tuyệt đối, vì cùng một công việc có thể được diễn đạt bằng nhiều câu khác nhau. Thay vào đó, hệ thống match theo trường `description` qua hai bước:

1. Tính điểm BM25 giữa mô tả task dự đoán và các mô tả task tham chiếu. Nếu điểm tốt nhất lớn hơn 2.0, cặp này được xem là match.
2. Nếu BM25 không đạt ngưỡng, hệ thống fallback sang Jaccard overlap trên tập token. Nếu overlap lớn hơn hoặc bằng 0.5, cặp này được xem là match.

Mỗi action item tham chiếu chỉ được match một lần. Một task dự đoán match thành công với một task tham chiếu chưa dùng được tính là true positive. Task dự đoán không match được tính là false positive. Task tham chiếu không được match bởi dự đoán nào được tính là false negative.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (5.1)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (5.2)$$

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (5.3)$$

Assignee accuracy được tính riêng sau khi description đã match. Nghĩa là nếu hệ thống trích xuất đúng nội dung công việc nhưng gán sai người phụ trách, task đó vẫn đóng góp vào true positive ở mức description, nhưng làm giảm assignee accuracy. Cách tách này giúp phân biệt hai lỗi khác nhau: bỏ sót/sinh thừa task và gán sai người thực hiện.

Schema failure rate đo tỷ lệ mẫu mà pipeline không trả được output hợp lệ, ví dụ lỗi parse JSON hoặc lỗi validate schema. Hallucination rate đo tỷ lệ task dự đoán không có bằng chứng đủ mạnh trong transcript; heuristic hiện tại kiểm tra overlap giữa các content words trong mô tả task và toàn bộ transcript. Đây là tín hiệu tự động để phát hiện regression, không thay thế hoàn toàn đánh giá thủ công.

5.3 Kết quả định lượng

Bảng 5.2 trình bày kết quả baseline của Qwen2.5-3B trên 100 mẫu của benchmark synthetic.

Metric	Kết quả	Vai trò trong đánh giá
Precision	0.8604	Hard gate: yêu cầu tối thiểu 0.70 để tránh sinh quá nhiều task sai.
Recall	0.6665	Watch metric: theo dõi mức độ bỏ sót task.
F1	0.6886	Watch/relative metric: candidate không được giảm quá 0.05 so với baseline.
Assignee accuracy	0.5232	Watch metric: đánh giá khả năng gán đúng người phụ trách.
Hallucination rate	0.0%	Hard gate: không được tăng quá 2 điểm phần trăm so với baseline.
Schema failure rate	0.0%	Hard gate: không được regression.
Average latency	26.97s	Watch metric: thời gian xử lý trung bình trên môi trường local.
P95 latency	120.2s	Watch metric: độ trễ ở nhóm mẫu chậm nhất.

Table 5.2: Kết quả baseline trên benchmark 100 mẫu

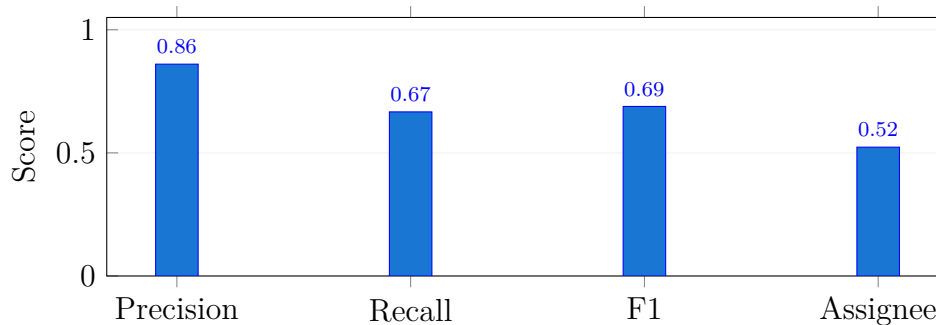


Figure 5.1: Các metric chất lượng chính của baseline

Kết quả cho thấy baseline có precision tốt: phần lớn task được sinh ra có mô tả khớp với action items tham chiếu. Schema failure rate bằng 0.0% cho thấy output JSON và lớp validate đủ ổn định trên tập benchmark này. Hallucination rate bằng 0.0% cho thấy guardrails và prompt hiện tại chưa tạo ra task hoàn toàn không có dấu vết trong transcript, theo heuristic đang dùng.

Hai điểm yếu chính là recall và assignee accuracy. Recall 0.6665 nghĩa là hệ thống vẫn bỏ sót một phần action items tham chiếu, đặc biệt với những task được nói gián tiếp

hoặc không có cấu trúc mệnh lệnh rõ ràng. Assignee accuracy 0.5232 cho thấy bài toán gán người phụ trách còn khó: người nói có thể giao việc cho người khác, tự nhận việc, hoặc nhắc lại một công việc đã thống nhất trước đó. Đây là nhóm lỗi cần ưu tiên trong feedback loop và prompt/model improvement.

5.4 Đánh giá độ trễ

Average latency 26.97 giây và P95 latency 120.2 giây phản ánh chi phí chạy Qwen2.5-3B qua Ollama trong môi trường local. Độ trễ này chưa phù hợp cho trải nghiệm real-time, nhưng chấp nhận được với workflow xử lý hậu kỳ sau cuộc họp: người dùng upload audio, đợi job hoàn tất, sau đó review kết quả.

P95 cao hơn trung bình nhiều cho thấy thời gian xử lý phụ thuộc mạnh vào độ dài transcript, số chunk cần gọi LLM và trạng thái tài nguyên máy chạy Docker. Trong bản hiện tại, latency được xem là watch metric chứ không phải hard gate. Nếu triển khai cho người dùng thật, cần bổ sung queue monitoring, giới hạn kích thước audio, batch/chunk strategy tốt hơn, hoặc phục vụ model trên phần cứng chuyên dụng.

5.5 Kiểm thử chức năng và vận hành

Bên cạnh benchmark định lượng, dự án có các kiểm thử chức năng để đảm bảo những phần chính không bị regression:

- Unit tests cho API, feedback loop, export dữ liệu, PII masking, anomaly detection và model promotion.
- Kiểm tra schema và guardrails để đảm bảo output LLM có thể lưu vào database.
- Docker Compose cho profile mặc định và profile `mlops`, gồm API, worker, Redis, PostgreSQL, Ollama, Prometheus và Grafana.
- Health check và metrics endpoint để xác nhận service có thể được quan sát khi chạy.

Các kiểm thử này không thay thế benchmark chất lượng mô hình, nhưng giúp đảm bảo hệ thống vẫn vận hành được khi thay đổi code. Với một hệ thống LLMOps, điều này quan trọng vì lỗi có thể đến từ nhiều lớp: prompt, schema, model serving, database, queue hoặc frontend feedback.

5.6 Phân tích kết quả

Kết quả baseline phù hợp với kỳ vọng của một mô hình base chưa fine-tune. Mô hình có xu hướng thận trọng hơn là sinh quá nhiều task: precision cao hơn recall. Đây là lựa chọn an toàn cho ứng dụng quản lý công việc, vì task sai có thể gây nhiễu trực tiếp cho người dùng. Tuy nhiên, recall thấp làm giảm giá trị tự động hóa vì người dùng vẫn phải đọc lại transcript để tìm việc bị bỏ sót.

Assignee accuracy là chỉ số đáng chú ý nhất. Trong meeting nhiều người, người nói và người chịu trách nhiệm không luôn trùng nhau. Ví dụ, một manager có thể nói “Bob, please send the report”, hoặc một engineer có thể nói “I will fix it”. Hai trường hợp này đòi hỏi hệ thống hiểu quan hệ giữa hành động, người được gọi tên, đại từ và speaker hiện tại. Vì vậy, assignee resolution cần kết hợp transcript context, speaker mapping, roster aliases và feedback từ người dùng.

Kết quả hallucination và schema failure bằng 0.0% là tín hiệu tốt, nhưng không nên diễn giải quá mức. Hallucination trong benchmark đang được đo bằng heuristic overlap nội dung, nên chỉ phát hiện các task gần như không có bằng chứng trong transcript. Những lỗi tinh vi hơn như sai deadline, sai priority, hoặc diễn giải quá rộng vẫn cần đánh giá thủ công và feedback UI.

5.7 Giới hạn của đánh giá

Đánh giá hiện tại có bốn giới hạn chính:

- Tập benchmark là synthetic, chưa phản ánh đầy đủ nhiễu âm thanh, ngắt lời, tiếng lóng, code-switching và thói quen nói của người dùng thật.
- Benchmark dùng transcript có sẵn, nên chưa đo tác động của lỗi ASR và diarization lên task extraction.
- Matching BM25/Jaccard chỉ xấp xỉ tương đồng ngữ nghĩa; một số task đúng nhưng diễn đạt khác xa có thể bị tính sai, và ngược lại.
- Assignee accuracy chỉ kiểm tra exact match theo tên sau khi description đã match, chưa đánh giá các trường hợp nhiều bí danh hoặc người chưa có trong roster một cách đầy đủ.

Vì các giới hạn này, kết quả ở chương này nên được hiểu là baseline kỹ thuật để so sánh các phiên bản sau. Khi có dữ liệu meeting thật và feedback từ người dùng, cần mở rộng benchmark bằng tập đánh giá thủ công, thêm phân tích lỗi theo loại meeting, và đo end-to-end từ audio đến action items.

Chapter 6

Thách thức và hạn chế

Chương này tổng hợp các thách thức kỹ thuật còn tồn tại của hệ thống. Các hạn chế được trình bày theo hướng ảnh hưởng trực tiếp đến chất lượng xử lý, khả năng vận hành và mức độ sẵn sàng khi mở rộng từ demo kỹ thuật sang môi trường sử dụng thực tế.

6.1 Chất lượng audio và lỗi lan truyền trong pipeline

Meeting AI Agent là một pipeline nhiều bước: audio được chuyển thành transcript, transcript được gán speaker, sau đó LLM trích xuất action items. Vì vậy lỗi ở bước trước có thể lan sang bước sau. Nếu ASR nhận sai từ khóa quan trọng, action item có thể bị bỏ sót hoặc hiểu sai. Nếu diarization tách nhầm speaker, hệ thống có thể gán sai người phụ trách dù nội dung task được trích xuất đúng.

Trong báo cáo này, benchmark chính đánh giá extraction từ transcript tham chiếu, chưa đo đầy đủ ảnh hưởng của nhiễu audio, overlap speech hoặc micro chất lượng thấp. Đây là giới hạn quan trọng vì cuộc họp thật thường có ngắt lời, nói đè, âm nền, chuyển ngôn ngữ và cách nói không hoàn chỉnh.

6.2 Diarization và định danh người nói

Diarization chỉ trả về nhãn speaker ẩn danh như `SPEAKER_00` hoặc `SPEAKER_01`; nó không tự biết danh tính thật của người nói. Hệ thống hiện xử lý điểm này bằng speaker mapping trong UI: người dùng xem transcript evidence và map speaker ẩn danh sang participant thật hoặc worker trong roster.

Cách làm này phù hợp với phạm vi dự án vì tránh suy đoán danh tính người nói khi không có voice enrollment. Tuy nhiên, nó cũng tạo thêm bước review thủ công. Nếu muốn tự động hóa cao hơn, hệ thống cần dữ liệu giọng nói tham chiếu, chính sách quyền riêng tư rõ ràng và cơ chế xác nhận danh tính an toàn hơn.

6.3 Assignee resolution

Kết quả đánh giá cho thấy assignee accuracy còn thấp hơn các metric mô tả task. Nguyên nhân không chỉ nằm ở model, mà còn ở bản chất ngôn ngữ cuộc họp. Người nói có thể giao việc cho người khác, tự nhận việc bằng đại từ “T”, nhắc đến một người theo bí danh, hoặc nói một task đã được thống nhất từ trước. Các trường hợp này đòi hỏi hệ thống kết hợp speaker hiện tại, nội dung câu, roster, alias và ngữ cảnh lân cận.

Trong phiên bản hiện tại, assignee resolution đã có roster và speaker mapping hỗ trợ, nhưng chưa đủ mạnh để xử lý toàn bộ trường hợp mơ hồ. Đây là một hướng cải tiến cần ưu tiên vì task đúng nội dung nhưng sai người phụ trách vẫn gây tác động lớn trong workflow quản lý công việc.

6.4 Giới hạn của dữ liệu đánh giá

Tập đánh giá chính hiện là synthetic benchmark. Tập này hữu ích để kiểm tra regression vì có cấu trúc ổn định và đủ số lượng mẫu, nhưng chưa thay thế được meeting thật. Dữ liệu synthetic thường sạch hơn, có deadline rõ hơn và ít nhiễu hơn hội thoại thực tế.

Ngoài ra, matching bằng BM25/Jaccard chỉ xấp xỉ tương đồng giữa task dự đoán và task tham chiếu. Một số task đúng nhưng diễn đạt khác xa có thể bị đánh giá thấp; ngược lại, một số task trùng nhiều từ nhưng sai sắc thái có thể được match. Vì vậy, benchmark tự động cần được bổ sung bằng phân tích lỗi thủ công trên một tập nhỏ nhưng chất lượng cao.

6.5 Hiệu năng và tài nguyên tính toán

WhisperX, diarization và LLM inference đều tiêu tốn tài nguyên. Khi chạy trên máy cá nhân bằng Docker Compose, độ trễ phụ thuộc mạnh vào CPU/GPU, dung lượng RAM, độ dài transcript và số lần gọi LLM. Kết quả P95 latency cao cho thấy hệ thống chưa phù hợp với yêu cầu real-time.

Fine-tuning cũng có yêu cầu phần cứng riêng. Luồng fine-tuning bằng QLoRA đã được chuẩn bị ở mức code, Dockerfile, queue và tracking, nhưng việc chạy hiệu quả gần như cần GPU/CUDA. Do đó, báo cáo xem fine-tuning là năng lực mở rộng đã chuẩn bị, không xem nó là bằng chứng đã cải thiện baseline nếu chưa có artifact và benchmark sau huấn luyện.

6.6 Bảo mật và dữ liệu nhạy cảm

Hệ thống đã có PII masking dựa trên pattern trước khi đưa nội dung vào prompt hoặc log debug. Cơ chế này giúp giảm rủi ro lộ email, số điện thoại, URL, IP và một số định danh phổ biến. Tuy nhiên, dữ liệu cuộc họp có thể chứa thông tin nhạy cảm không theo pattern cố định, ví dụ tên khách hàng, chiến lược kinh doanh, thông tin nhân sự hoặc dữ kiện nội bộ.

Vì vậy, PII masking hiện tại không nên được mô tả như một hệ thống DLP hoàn chỉnh. Khi triển khai thực tế, cần bổ sung chính sách retention, phân quyền chi tiết, mã hóa secret, audit access và quy trình xử lý dữ liệu theo yêu cầu tổ chức.

6.7 Mức độ sẵn sàng triển khai

Hệ thống hiện chạy ổn cho demo kỹ thuật bằng Docker Compose. Cách triển khai này phù hợp để kiểm chứng pipeline, UI, database, worker và monitoring trên cùng một máy. Tuy nhiên, nó chưa tương đương môi trường production có autoscaling, backup, secret manager, CI/CD release gate, alerting đầy đủ và incident response.

Triển khai qua domain public hoặc tunnel có thể phục vụ demo ngắn hạn, nhưng chưa được xem là kết quả vận hành chính thức của báo cáo. Điểm đáng tin cậy của dự án ở thời điểm hiện tại là một hệ thống end-to-end có thể tái lập cục bộ, có benchmark baseline và có nền tảng LLMOps để tiếp tục cải tiến.

Chapter 7

Hướng phát triển

Các hướng phát triển dưới đây được sắp xếp theo mức độ ảnh hưởng đến chất lượng hệ thống. Ưu tiên chính không phải thêm nhiều tính năng rời rạc, mà là cải thiện độ tin cậy của pipeline NLP, mở rộng dữ liệu đánh giá và làm cho quy trình vận hành an toàn hơn.

7.1 Mở rộng dữ liệu đánh giá và feedback

Hướng quan trọng nhất là thu thập thêm dữ liệu feedback từ UI và xây dựng tập đánh giá nhỏ nhưng được kiểm chứng thủ công. Tập này nên bao gồm transcript thật hoặc transcript đã được scrub thông tin nhạy cảm, có action items tham chiếu, assignee, deadline và ghi chú về các trường hợp mơ hồ.

Dữ liệu mới nên được chia theo mục đích:

- Tập smoke test nhỏ để chạy nhanh trong CI.
- Tập benchmark chính để so sánh baseline và candidate model.
- Tập phân tích lỗi thủ công để hiểu các lỗi như missing task, wrong assignee, wrong due date và hallucination.

Khi dữ liệu feedback đủ lớn và đủ sạch, có thể dùng nó cho prompt tuning, fine-tuning hoặc preference-style training. Tuy nhiên, mọi candidate cần được đánh giá lại bằng cùng benchmark trước khi được xem là cải tiến.

7.2 Đánh giá audio end-to-end

Benchmark hiện tại tập trung vào extraction từ transcript. Để đánh giá đúng toàn bộ hệ thống audio meeting, cần bổ sung benchmark end-to-end từ audio đến action items. Benchmark này cần ít nhất ba loại annotation: transcript chuẩn, speaker segment/speaker identity và action items tham chiếu.

Khi có dữ liệu này, hệ thống có thể đo thêm:

- Word Error Rate cho ASR.
- Diarization Error Rate cho speaker diarization.
- Tác động của lỗi ASR/diarization đến precision, recall và assignee accuracy.
- Độ trễ theo thời lượng audio thay vì chỉ theo số sample transcript.

Đây là bước cần thiết nếu hệ thống được dùng trong môi trường có audio thật thay vì chỉ demo trên transcript hoặc audio kiểm soát.

7.3 Cải thiện assignee và deadline extraction

Assignee accuracy hiện là một trong các điểm yếu chính. Hướng cải tiến gồm chuẩn hóa alias trong roster, dùng speaker mapping làm tín hiệu mạnh hơn, mở rộng prompt với ví dụ về giao việc gián tiếp và bổ sung rule hậu xử lý cho các mẫu câu phổ biến như “I will...”, “Bob, please...”, hoặc “Can you...”.

Deadline extraction cũng cần được đánh giá riêng. Các cụm như “next Friday”, “end of next week” hoặc “before the release” phụ thuộc vào meeting date và ngữ cảnh nghiệp vụ. Một hướng phát triển tốt là tách due-date normalization thành module có test riêng, thay vì để toàn bộ trách nhiệm nằm trong LLM.

7.4 Nâng cấp evaluation harness

Evaluation harness hiện đã có matching BM25/Jaccard, assignee accuracy, hallucination heuristic và schema failure rate. Các cải tiến tiếp theo gồm:

- Thêm semantic matching bằng embedding hoặc cross-encoder để giảm sai lệch do diễn đạt khác nhau.
- Tách metric theo field: description, assignee, due date, priority và evidence turn.
- Báo cáo lỗi theo domain, số speaker, độ dài meeting và số action items.
- Lưu per-sample error cases để phục vụ phân tích sau benchmark.
- Chuẩn hóa promotion report để mỗi lần thay prompt/model đều có bảng chứng so sánh.

Mục tiêu của phần này là biến benchmark thành công cụ ra quyết định, không chỉ là bảng điểm tổng hợp.

7.5 Fine-tuning và model lifecycle

Khi có đủ dữ liệu correction chất lượng tốt và môi trường GPU phù hợp, có thể chạy QLoRA fine-tuning cho mô hình extraction. Candidate sau huấn luyện cần đi qua cùng evaluation gate: precision không được giảm dưới ngưỡng hard gate, hallucination/schema failure không regression, và F1 không giảm đáng kể so với baseline.

Sau khi candidate đạt gate, hệ thống có thể mở rộng theo hướng canary hoặc A/B testing. Một phần traffic nhỏ được route sang candidate, kết quả feedback và metrics được so sánh với baseline trong một khoảng thời gian. Chỉ khi candidate ổn định mới chuyển thành model mặc định.

7.6 Nâng cao vận hành

Để tiến gần hơn đến môi trường sử dụng thật, hệ thống cần các cải tiến vận hành sau:

- Reverse proxy HTTPS, domain cấu hình ổn định và secret manager.
- Backup/restore PostgreSQL và chính sách lưu trữ artifact.
- Role-based access control chi tiết hơn cho user/admin.
- Alert rules cho queue backlog, worker failure, API latency, LLM latency và disk usage.
- Retention policy cho audio, transcript, prompt trace và feedback data.
- Dashboard tách rõ metrics vận hành và metrics chất lượng mô hình.

Những cải tiến này không thay đổi thuật toán NLP cốt lõi, nhưng quyết định khả năng hệ thống hoạt động ổn định khi có nhiều người dùng và dữ liệu thật.

7.7 Hỗ trợ tiếng Việt và đa ngôn ngữ

Dự án hiện ưu tiên pipeline tiếng Anh trong benchmark. Nếu mở rộng cho người dùng Việt Nam, cần bổ sung dữ liệu tiếng Việt, hội thoại trộn Anh-Việt, cách gọi tên người Việt và cách diễn đạt deadline trong tiếng Việt. Đây không chỉ là bài toán dịch ngôn ngữ; nó ảnh hưởng đến ASR, diarization review, prompt, assignee resolution và evaluation.

Hướng phù hợp là xây dựng một tập đánh giá tiếng Việt nhỏ trước, đo baseline, sau đó mới quyết định cần prompt tuning, fine-tuning hay thay đổi ASR/model.

Chapter 8

Kết luận

Đề tài đã xây dựng Meeting AI Agent như một hệ thống NLP end-to-end cho bài toán xử lý audio cuộc họp và trích xuất action items. Hệ thống bao gồm các thành phần chính: upload audio, speech-to-text bằng WhisperX, phân tách speaker, chuẩn hóa transcript theo lượt nói, trích xuất summary/action items bằng LLM, kiểm tra output bằng guardrails, lưu trữ dữ liệu vào PostgreSQL, giao diện human feedback, monitoring bằng Prometheus/Grafana và nền tảng LLMOps cho đánh giá/fine-tuning sau này.

Về mặt kỹ thuật, đóng góp chính của dự án không chỉ nằm ở việc gọi một mô hình ngôn ngữ lớn, mà ở cách tổ chức toàn bộ vòng xử lý. Hệ thống tách rõ frontend, API, worker, database, queue, model serving và observability. Các action items được lưu cùng metadata, source turns và feedback corrections, giúp người dùng truy vết kết quả thay vì chỉ nhận một danh sách task thiếu ngữ cảnh. Đây là điểm quan trọng đối với ứng dụng doanh nghiệp, nơi khả năng giải thích và sửa lỗi thường quan trọng không kém độ chính xác ban đầu.

Kết quả baseline với Qwen2.5-3B trên 100 mẫu của `data/eval/gold_synthetic_205.jsonl` đạt precision 0.8604, recall 0.6665, F1 0.6886, assignee accuracy 0.5232, hallucination rate 0.0% và schema failure rate 0.0%. Các con số này cho thấy pipeline đã ổn định về định dạng output và có precision tốt ở bước trích xuất task. Tuy nhiên, recall và assignee accuracy vẫn là điểm cần cải thiện, đặc biệt khi chuyển từ dữ liệu synthetic sang meeting thật.

Dự án cũng đã chuẩn bị các thành phần cần thiết cho LLMOps: evaluation harness, benchmark baseline, feedback export, retrain trigger, MLflow tracking và quy trình promotion có kiểm soát. Tuy vậy, báo cáo phân biệt rõ giữa năng lực đã chuẩn bị và kết quả đã hoàn tất. Fine-tuning chưa được xem là bằng chứng cải thiện nếu chưa có artifact, benchmark sau huấn luyện và so sánh với baseline.

Tài liệu tham khảo

- [1] Microsoft Corporation. *Microsoft Work Trend Index: Hybrid Work Is Just Work*. Microsoft, 2022. <https://www.microsoft.com/en-us/worklab/work-trend-index>
- [2] M. Bain, J. Huh, T. Han, and A. Zisserman. WhisperX: Time-accurate speech transcription of long-form audio. *arXiv preprint arXiv:2303.00747*, 2022. <https://arxiv.org/abs/2303.00747>
- [3] H. Bredin et al. pyannote.audio 2.1 speaker diarization: Principle, benchmark, and recipe. In *Proceedings of Interspeech 2023*, 2023. <https://huggingface.co/pyannote/speaker-diarization-3.1>
- [4] Qwen Team. *Qwen2.5 Technical Report*. Alibaba Cloud, 2024. <https://arxiv.org/abs/2412.15115>
- [5] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. <https://arxiv.org/abs/2305.14314>
- [6] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. <https://arxiv.org/abs/2106.09685>
- [7] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever. Robust speech recognition via large-scale weak supervision. In *International Conference on Machine Learning (ICML)*, 2023. <https://arxiv.org/abs/2212.04356>
- [8] J. Johnson, M. Douze, and H. Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019. <https://github.com/facebookresearch/faiss>
- [9] V. Shankar, R. Garcia, J. E. Gonzalez, and J. Hellerstein. Operationalizing machine learning: An interview study. *arXiv preprint arXiv:2209.09125*, 2022. <https://arxiv.org/abs/2209.09125>
- [10] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, A. Madotto, and P. Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023. <https://arxiv.org/abs/2202.03629>

- [11] T. B. Brown et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. <https://arxiv.org/abs/2005.14165>
- [12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Aiden, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. <https://arxiv.org/abs/1706.03762>
- [13] The Prometheus Authors. *Prometheus Monitoring System & Time Series Database*. <https://prometheus.io>, 2024.
- [14] Celery Project. *Celery: Distributed Task Queue*. <https://docs.celeryq.dev>, 2024.
- [15] S. Ramírez. *FastAPI Framework, high performance, easy to learn, fast to code*. <https://fastapi.tiangolo.com>, 2024.

Appendix A

Cấu trúc dự án hiện tại

```
1 web/                                # Next.js frontend
2 src/meeting_agent/
3   api/main.py                       # FastAPI endpoints
4   db/models.py                      # SQLAlchemy ORM schema
5   pipeline/
6     stt.py                          # WhisperX + diarization
7     orchestrator.py                 # Summary/action
6     extraction_pipeline
8     worker_task.py                  # Celery tasks and queue
6     routing
9     feedback.py                     # Human feedback
6     persistence
10  mlops/
11    evaluate.py                     # Evaluation harness
12    finetune.py                     # GPU fine-tune
6    entrypoint
13    data_pipeline/                  # Feedback/data export
6    helpers
14  scripts/deploy_promoted_model.py   # Controlled deploy flow
15  scripts/run_benchmark.py           # Baseline/candidate
6    benchmark
16  scripts/prepare_hf_dataset.py      # Hugging Face dataset
6    export
17  docker-compose.yml
18  Dockerfile.train
19  Makefile
20  docs/mlops-runbook.md
21  docs/project-status.md
22  data/eval/gold_synthetic_205.jsonl
23  data/training/
```

Listing A.1: Các thư mục và file chính

Appendix B

Hướng dẫn chạy

B.1 Chạy môi trường mặc định

```
1 docker compose up -d --build
```

Listing B.1: Khởi động stack chính

Các URL thường dùng:

- Frontend: `http://localhost:3001`
- API health: `http://localhost:8000/health`
- Prometheus metrics: `http://localhost:8000/metrics`
- Prometheus UI: `http://localhost:9090`
- Grafana: `http://localhost:3000`

B.2 Chạy profile MLOps

```
1 make mlops-up
2 make retrain-check
3 make retrain-force
4 make train-image
```

Listing B.2: Khởi động các service MLOps

Profile này bổ sung Celery Beat, trainer worker và MLflow. Trainer consume queue `mlops`, tách khỏi worker xử lý meeting bình thường.

B.3 Deploy model promoted

```
1 make deploy-promoted-model APPLY=1
```

Listing B.3: Deploy model promoted có chủ đích

Deploy không tự động chạy ngay sau promotion. Operator phải thực hiện lệnh deploy để giữ audit trail và giảm rủi ro thay model ngoài ý muốn.

Appendix C

Ví dụ API

C.1 Upload meeting

```
1 curl -X POST http://localhost:8000/meetings \
2   -F "audio=@meeting.wav" \
3   -F 'roster_json={
4     "workers": [
5       {
6         "worker_id": "w1",
7         "name": "Alice Chen",
8         "aliases": ["Alice"],
9         "role": "Product Manager"
10      },
11      {
12        "worker_id": "w2",
13        "name": "Bob Kim",
14        "aliases": ["Bob"],
15        "role": "Backend Engineer"
16      }
17    ]
18  },'
```

Listing C.1: Upload audio

C.2 Lấy trạng thái meeting

```
1 curl http://localhost:8000/meetings/{meeting_id}
```

Listing C.2: Xem trạng thái job

C.3 Gửi feedback

```
1 curl -X POST
   http://localhost:8000/meetings/{meeting_id}/feedback \
```

```
2  -H "Content-Type: application/json" \  
3  -d '{  
4      "reviewer": "human-reviewer",  
5      "notes": "Assignee corrected after transcript review",  
6      "corrections": [  
7          {  
8              "task_id": "task-1",  
9              "original_description": "Send the quarterly report",  
10             "corrected_description": "Send the quarterly report to  
11                 the client",  
12             "original_assignee": "Bob",  
13             "corrected_assignee": "Alice Chen",  
14             "original_due_date": null,  
15             "corrected_due_date": "2026-05-30",  
16             "is_false_positive": false,  
17             "is_missing": false  
18         }  
19     ]  
    },'
```

Listing C.3: Gửi correction từ human review

C.4 Xuất feedback cho fine-tuning

```
1 curl "http://localhost:8000/feedback/export?format=jsonl"
```

Listing C.4: Export feedback dạng JSONL

Appendix D

Cấu hình môi trường quan trọng

Biến	Mục đích
DATABASE_URL	Kết nối PostgreSQL
REDIS_URL	Redis broker/cache
OLLAMA_BASE_URL	Endpoint Ollama
OLLAMA_LLM_MODEL	Model LLM đang phục vụ
WHISPER_MODEL	Model WhisperX
WHISPER_DEVICE	cpu hoặc cuda
HF_TOKEN	Token cho model diarization nếu cần
WEB_PORT	Port public của frontend, mặc định 3001
PUBLIC_URL	URL public/frontend dùng khi cấu hình domain hoặc tunnel
AUTH_URL	URL callback cho NextAuth
GOOGLE_CLIENT_ID	OAuth client ID cho đăng nhập Google
GOOGLE_CLIENT_SECRET	OAuth client secret cho đăng nhập Google
LANGCHAIN_TRACING_V2	Bật/tắt LangSmith tracing nếu có cấu hình
LANGCHAIN_API_KEY	API key LangSmith, nếu dùng
RETRAIN_MIN_CORRECTIONS	Ngưỡng correction để retrain check
MLFLOW_TRACKING_URI	Tracking URI cho MLflow

Các giá trị bí mật phải đặt trong `.env`; `.env.example` chỉ giữ tên biến và giá trị mẫu không nhạy cảm.